

TP RL

Table des matières

1 Environnement de code / configuration	1
2 Algorithme DQN	2
2.1 Algorithme	2
2.2 CartPole-v1	4
2.2.1 Sans réseau cible ni prioritized experience replay	4
2.2.2 Prioritized experience replay	4
2.2.3 Réseau cible et prioritized experience replay	7
2.3 LunarLander-v3	7
2.3.1 Sans réseau cible ni prioritized experience replay	8
2.4 Avec réseau cible et prioritized experience replay	9
2.5 Double DQN	10

Le code est disponible ici: <https://github.com/arnaud-ma/RL-TP>

1 Environnement de code / configuration

L'utilisation de uv simplifie les choses:

- s'installe rapidement en une commande (<https://docs.astral.sh/uv/#installation>)
- `uv init --package --python 3.13` crée la structure du projet
- `uv add <package>` ajoute les dépendances. Le fichier `pyproject.toml` qui contient toutes les métadonnées du projet (y compris les dépendances) est automatiquement mis à jour.
- `uv run <script>` pour lancer un script python. Cette dernière commande s'occupe, quelle que soit la machine, de automatiquement:
 - installer la bonne version de Python (si besoin)
 - créer un environnement virtuel (si besoin)
 - activer l'environnement virtuel (si besoin)
 - installer les dépendances (si besoin)
 - lancer le script

Cela permet d'avoir la sécurité d'avoir un environnement par projet, tout en n'ayant pas besoin de le gérer soi-même du tout. Il y a une alternative `pixi` qui rajoute par défaut une compatibilité avec `conda`, mais dans notre cas `uv` suffit puisqu'on utilise que des paquets venant de Python.

Pour charger les configurations, j'utilise `pydantic`.

```
class ConfigModel(BaseModel):
    env: str

    seed: PositiveInt
    device: str | int
    render_mode: Literal["human", "terminal", "both"] = "both"
    deterministic_config: bool = False

    max_length_train: PositiveInt
    max_length_test: PositiveInt

    epsilon_start: ZeroToOne
    epsilon_min: ZeroToOne
```

epsilon_decay: ZeroToOne

...

Cela permet de facilement valider les configurations, directement avoir une erreur si une valeur est incorrecte, documenter les paramètres, et avoir un vrai objet python à la place d'un dictionnaire (plus pratique à utiliser, auto-complétion, etc.).

2 Algorithme DQN

2.1 Algorithme

```
1  Données :
2  | Un environnement de Markov avec états S, actions A, récompenses R
3  | Un taux d'actualisation  $\gamma \in [0, 1]$ 
4  | Un taux d'exploration initial  $\varepsilon \in [0, 1]$ 
5  | Un taux d'exploration minimum  $\varepsilon_{\min} \in [0, \varepsilon]$ 
6  | Un facteur de décroissance de l'exploration  $\varepsilon_{\text{decay}} \in (0, 1)$ 
7  | Une capacité de mémoire  $N$ 
8  | Une taille de mini-lot  $M$ 
9  | Un nombre maximum d'étapes par épisode  $T_{\max}$ 
10 | Une fonction de perte  $l$  (ex: erreur quadratique)
11 un optimiseur de descente de gradient  $O$  (ex: Adam, RMSProp)
12 | Une fréquence d'optimisation  $F$ 
13 | Une fréquence de mise à jour du réseau cible  $C$ 
14 Résultat : Les poids  $\theta$  du réseau de Q-valeurs appris.
15 Initialisation :
16 | la mémoire de replay  $D$  à capacité  $N$ 
17 | le réseau de Q-valeurs  $Q_\theta$  avec des poids aléatoires  $\theta$ 
18 | le réseau cible  $Q_{\theta^-}$  avec des poids  $\theta^- = \theta$ 
19 Boucle principale :
20 | pour chaque épisode faire
21 | | Initialise l'état initial  $s_0$ 
22 | | pour  $t = 0$  à  $T_{\max}$  et tant que  $s_t$  n'est pas terminal faire
23 | | |  $a_t \leftarrow \begin{cases} a \sim \mathcal{U}(A) \text{ avec probabilité } \varepsilon \\ \arg\max_a Q_\theta(s_t, a) \text{ sinon} \end{cases}$ 
24 | | |  $\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon * \varepsilon_{\text{decay}})$ 
25 | | | exécute l'action  $a_t$ , observe la récompense  $r_t$  et le nouvel état  $s_{t+1}$ 
26 | | | stocke la transition  $(s_t, a_t, r_t, s_{t+1})$  dans la mémoire de replay  $D$ 
27 | | si la taille de  $D \geq M$  et  $t \bmod F = 0$  alors
28 | | | échantillonne un mini-lot de  $M$  transitions  $(s_j, a_j, r_j, s_{j+1})$  de  $D$  uniformément,
29 | | | sans remise
30 | | | pour chaque transition  $j$  du mini-lot faire
31 | | | |  $V_j \leftarrow \begin{cases} 0 \text{ si } s_{j+1} \text{ est terminal} \\ \max_{a'} Q_{\theta^-}(s_{j+1}, a') \text{ sinon} \end{cases}$ 
32 | | | |  $y_j \leftarrow r_j + \gamma V_j$ 
33 | | | | On note  $l_j : \theta \mapsto l(Q_\theta(s_j, a_j), y_j)$ 
```

```

33 | | | fin pour
34 | | | On note  $L : \theta \mapsto \frac{1}{M} \sum_j l_j(\theta)$ 
35 | | |  $\theta \leftarrow \text{O.step}(\theta, \nabla_\theta L(\theta))$  (descente de gradient)
36 | | | si  $t \bmod C = 0$ 
37 | | | |  $\theta^- \leftarrow \theta$  (copie des poids du réseau de Q-valeurs vers le réseau cible)
38 | | | fin si
39 | | fin pour (action)
40 | fin pour (épisode)

```

Ensuite pour évaluer la politique apprise:

```

1 Données :
2 | Le réseau de Q-valeurs appris  $Q_\theta$ 
3 | Un nombre maximum d'étapes  $T_{\max}$ 
4 Résultat : La récompense  $R$ .
5 Initialise l'état initial  $s_0$ 
6  $R \leftarrow 0$ 
7 pour  $t = 0$  à  $T_{\max}$  et tant que  $s_t$  n'est pas terminal faire
8 |  $a_t \leftarrow \operatorname{argmax}_a Q_\theta(s_t, a)$ 
9 | exécute l'action  $a_t$ , observe la récompense  $r_t$  et le nouvel état  $s_{t+1}$ 
10 |  $R \leftarrow R + r_t$ 
11 fin pour

```

- On peut faire une version sans réseau cible: il suffit de remplacer tous les θ^- par θ dans l'algorithme.
- On peut faire une version sans mémoire de replay: il suffit de remplacer l'échantillonnage de mini-lots par l'utilisation de la transition courante. Le mini-lot est alors de taille 1.
- Une version avec une mise à jour douce du réseau cible peut être faite en remplaçant la copie des poids par une interpolation:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

avec $\tau \in (0, 1]$ un petit paramètre (ex: 0.001), qui remplace la fréquence de mise à jour C .

- Une version avec prioritized experience replay peut être faite en modifiant la manière d'échantillonner les transitions dans la mémoire de replay, et en ajoutant des poids d'importance dans le calcul de la perte:
 1. Lors de l'échantillonnage des transitions, utiliser une probabilité proportionnelle à l'erreur TD ($l(Q_\theta(s_j, a_j), y_j)$).
 2. Calculer des poids d'importance pour chaque transition échantillonnée.
 3. Utiliser ces poids dans le calcul de la perte:

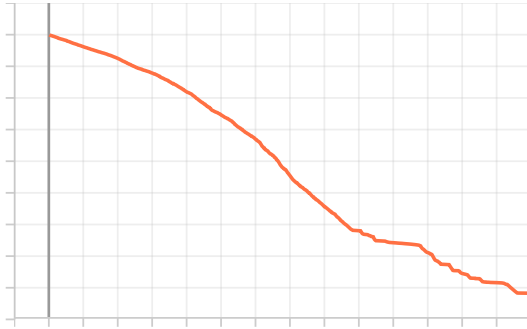
$$L(\theta) = \frac{1}{M} \sum_j w_j l(Q_\theta(s_j, a_j), y_j)$$

- Une version Double DQN peut être faite en modifiant le calcul de V_j :

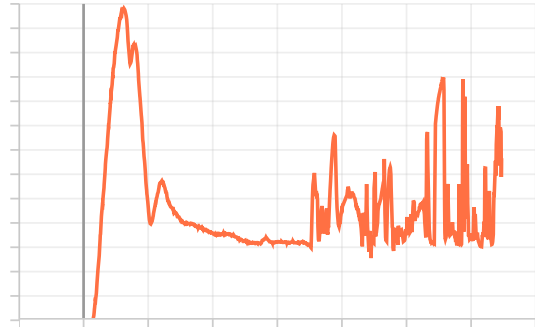
$$V_j \leftarrow \begin{cases} 0 & \text{si } s_{j+1} \text{ est terminal} \\ Q_{\theta^-}(s_{j+1}, \operatorname{argmax}_{a'} Q_{\theta}(s_{j+1}, a')) & \text{sinon} \end{cases}$$

2.2 CartPole-v1

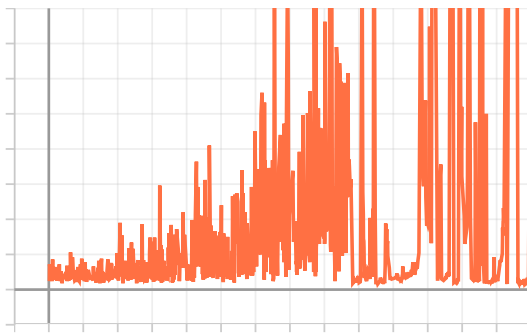
2.2.1 Sans réseau cible ni prioritized experience replay



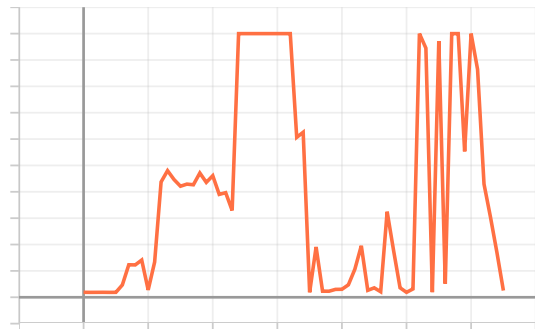
(a) ϵ en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



(c) Récompense obtenue lors des épisodes d'entraînement



(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 1. – Résultats de l'entraînement du DQN sans réseau cible ni prioritized experience replay sur l'environnement CartPole-v1.

Je ne remarque pas de grandes différences si on rajoute un réseau cible. L'apprentissage reste peu stable, et la variance entre différentes trajectoires d'entraînement est importante. Par exemple dans l'exemple ci-dessus, on a eu de la chance d'atteindre une bonne politique assez rapidement.

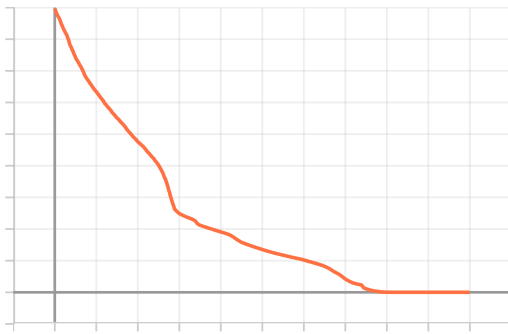
2.2.2 Prioritized experience replay

```
ConfigDQN(
    env='CartPole-v1',
    seed=6,
    max_length_train=500,
    max_length_test=500,
    epsilon_start=0.9,
    epsilon_min=0.0001,
    epsilon_decay=0.9999,
    gamma=0.99,
    learning_rate=0.0003,
    batch_size=100,
```

```

optimizer=<class 'torch.optim.adam.Adam'>,
value_loss=<class 'torch.nn.modules.loss.SmoothL1Loss'>,
train_freq=1,
use_target_network=False,
clip_grad_norm=10,
hidden_layers=[200],
hidden_layers_activation=<class 'torch.nn.modules.activation.Tanh'>,
final_layer_activation=<class 'torch.nn.modules.linear.Identity'>,
replay_memory_capacity=10000,
replay_memory_min_size=500,
prioritized_replay=True,
nb_episodes=2000,
freq_test=50,
nb_tests=100,
)

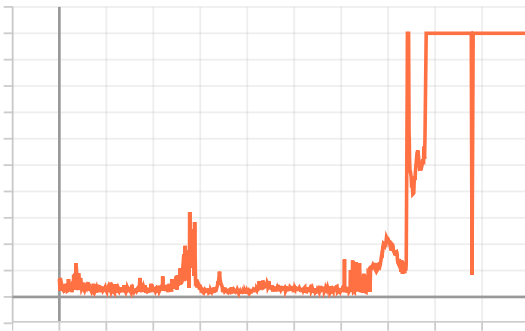
```



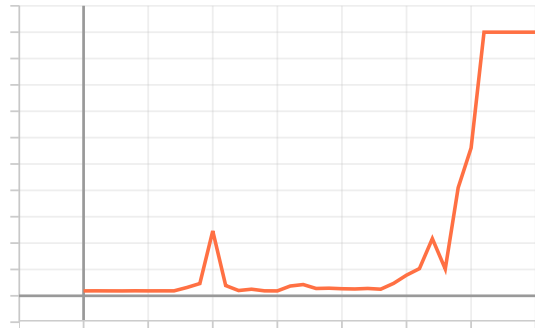
(a) ε en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



(c) Récompense obtenue lors des épisodes d'entraînement



(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 2. – Résultats de l'entraînement du DQN sur l'environnement CartPole-v1.

La Q-valeur se stabilise vers 100. Ce qui confirme la théorie:

$$\begin{aligned}
Q^*(s, a) &= \mathbb{E}[R_0 \mid s_0 = s, a_0 = a] \\
&= \mathbb{E} \left[\sum_{i=0}^{T-1} \gamma^i r_i \mid s_0 = s, a_0 = a \right] \\
&= \sum_{i=0}^{T-1} \gamma^i \mathbb{E}[r_i \mid s_0 = s, a_0 = a]
\end{aligned}$$

Et si notre politique est bonne, quel que soit l'état s et l'action a , on a une récompense proche de 1 en moyenne à chaque étape, donc:

$$Q^*(s, a) \approx \sum_{i=0}^{T-1} \gamma^i = \frac{1 - \gamma^T}{1 - \gamma} \approx \frac{1}{1 - \gamma}$$

On remarque que la Q-valeur approximée par le réseau est souvent plus grande que cette valeur théorique. On peut faire un lien avec la loss lors de l'apprentissage: si la Q-valeur est sur-estimée, alors les cibles y_j seront aussi sur-estimées, et donc la loss sera plus grande.

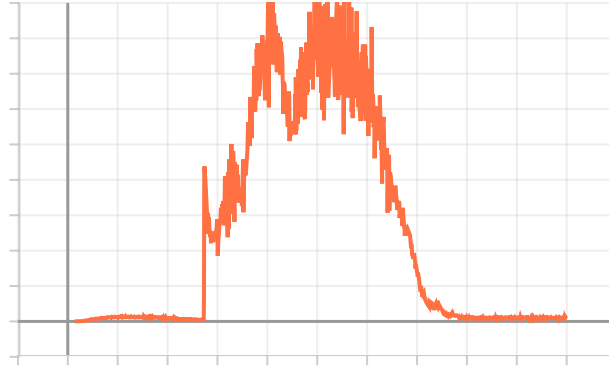
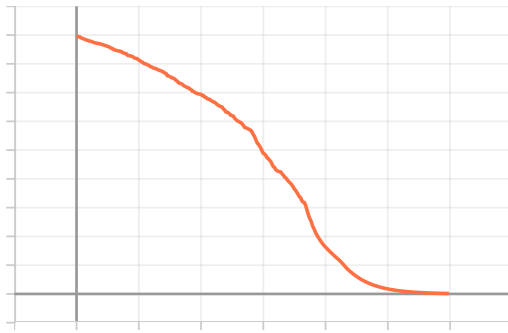


Fig. 3. – Loss de l'entraînement du DQN sur l'environnement CartPole-v1.

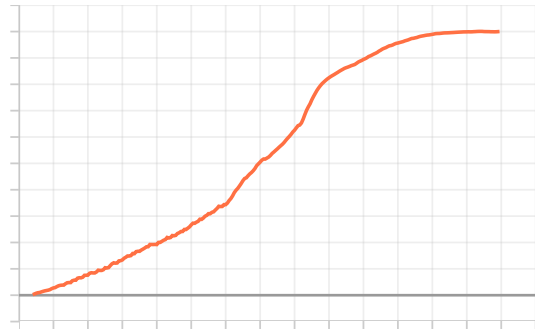
Je n'ai pas remarqué de grande différence avec ou sans prioritized experience replay sur cet environnement.

De même, on pourrait penser que le Double DQN permettrait de réduire cette sur-estimation, mais dans les faits je n'ai pas remarqué de différence.

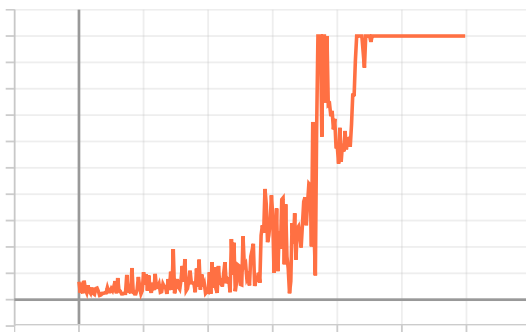
2.2.3 Réseau cible et prioritized experience replay



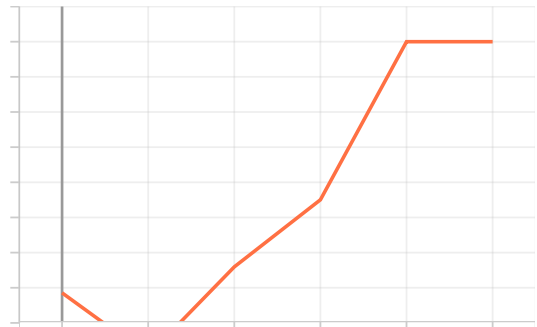
(a) ϵ en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



(c) Récompense obtenue lors des épisodes d'entraînement



(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 4. – Résultats de l'entraînement du DQN avec réseau cible et prioritized experience replay sur l'environnement CartPole-v1.

L'apprentissage est bien plus sample-efficace. Seulement 300 épisodes contre 16000 de celle juste au-dessus pour atteindre une bonne politique.

2.3 LunarLander-v3

L'environnement Lunar étant plus complexe, on change:

- la taille du replay buffer à 100000 (si utilisé).
- la taille du réseau à deux couches (128, 128)
- la fonction d'activation des couches cachées à ReLU pour accélérer l'apprentissage
- on change l'optimiseur à AdamW, une variante d'Adam.
- mise à jour du réseau cible tous les 1000 événements d'entraînement au lieu de tous les 100.
- Diminution de la taille du batch à 64, pour gagner du temps de calcul.

Et bien sûr, une adaptation de ϵ pour que l'exploration dure plus longtemps.

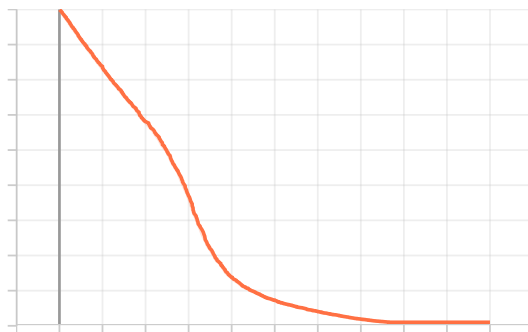
```
ConfigDQN(  
    env='LunarLander-v3',  
    seed=6,  
    nb_episodes=10000,  
    freq_test=100,  
    nb_tests=2,  
    max_length_train=1000,
```

```

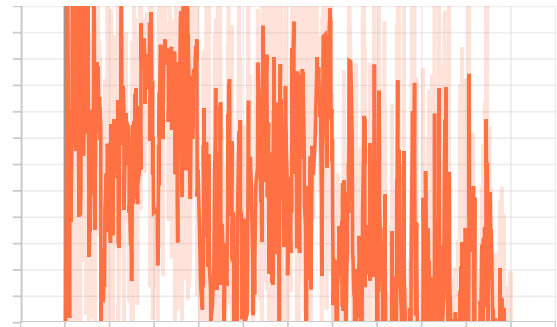
max_length_test=1000,
epsilon_start=0.9,
epsilon_min=0.01,
epsilon_decay=0.999998,
gamma=0.99,
learning_rate=0.0003,
batch_size=64,
optimizer=<class 'torch.optim.adamw.AdamW'>,
value_loss=<class 'torch.nn.modules.loss.SmoothL1Loss'>,
train_freq=4,
use_target_network=False,
target_update_freq=1000,
target_soft_update=0.0,
clip_grad_norm=0,
hidden_layers=[128, 128],
hidden_layers_activation=<class 'torch.nn.modules.activation.ReLU'>,
final_layer_activation=<class 'torch.nn.modules.linear.Identity'>,
dropout=0.0,
replay_memory_capacity=0,
replay_memory_min_size=1000,
prioritized_replay=False,
)

```

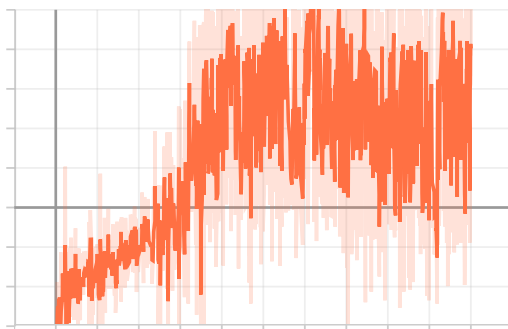
2.3.1 Sans réseau cible ni prioritized experience replay



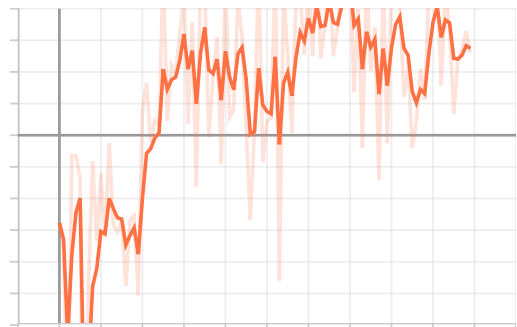
(a) ϵ en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



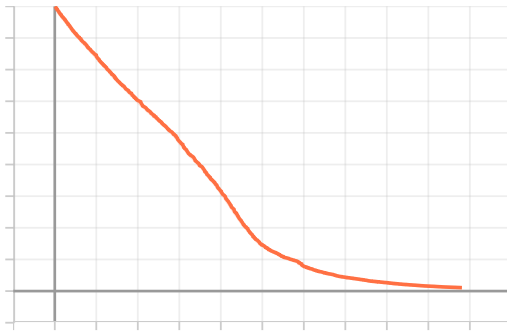
(c) Récompense obtenue lors des épisodes d'entraînement



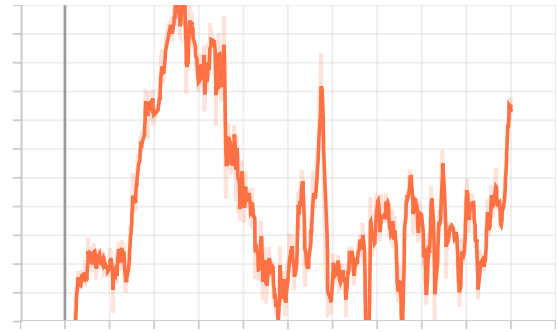
(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 5. – Résultats de l'entraînement du DQN sans réseau cible ni prioritized experience replay sur l'environnement LunarLander-v3.

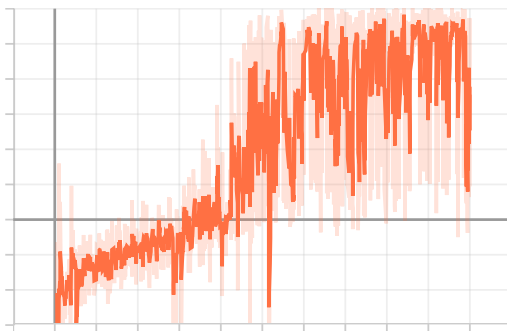
2.4 Avec réseau cible et prioritized experience replay



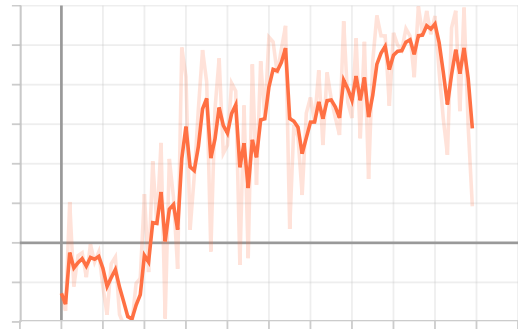
(a) ϵ en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



(c) Récompense obtenue lors des épisodes d'entraînement



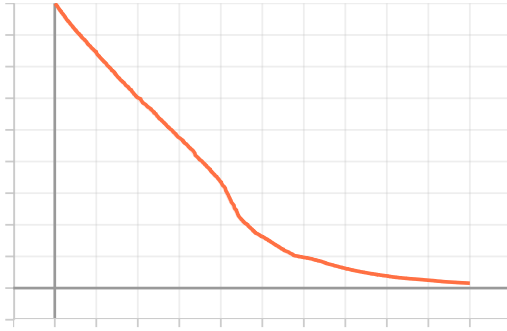
(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 6. – Résultats de l'entraînement du DQN avec réseau cible et prioritized experience replay sur l'environnement LunarLander-v3.

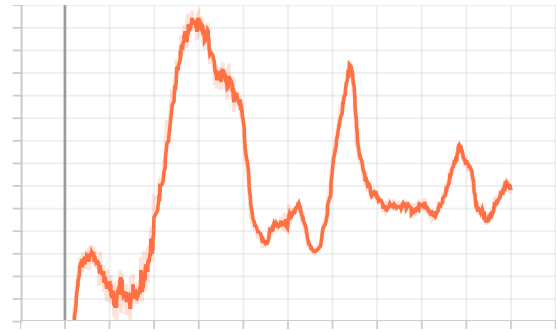
Même avec le réseau cible et le prioritized experience replay, l'apprentissage la Q-valeur ne semble pas stable (même si bien meilleur que sans).

Testons avec le Double DQN.

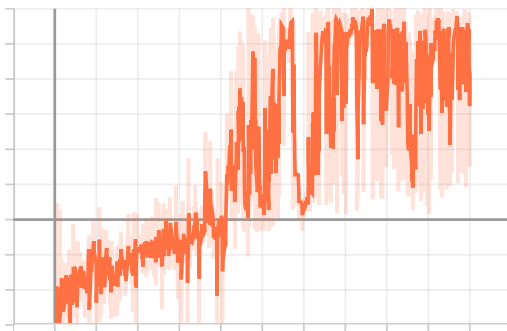
2.5 Double DQN



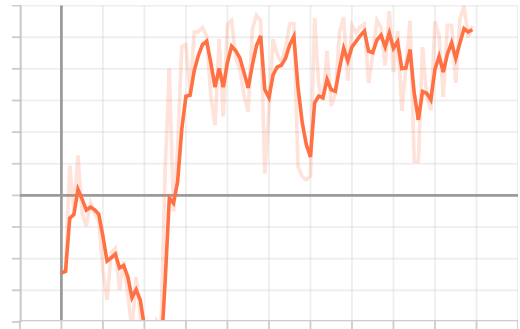
(a) ϵ en fonction des épisodes d'entraînement.



(b) Q-valeur moyenne des états visités en fonction des épisodes d'entraînement.



(c) Récompense obtenue lors des épisodes d'entraînement



(d) Récompense moyenne obtenue de 100 épisodes de test tous les 50 épisodes d'entraînement.

Fig. 7. – Résultats de l'entraînement du Double DQN avec réseau cible et prioritized experience replay sur l'environnement LunarLander-v3.

Le Double DQN permet de stabiliser l'apprentissage de la Q-valeur, et donc de la politique. Il faudrait avoir un temps d'entraînement plus long.